

Lesson 5: Classical Deep Learning with PyTorch

Lesson 5: Classical Deep Learning with PyTorch CNN, MNIST, training, loss, optimization, and the hybrid-ML bridge

Dr. Syed Muzamil Basha

School of Computer Science and Engineering, REVA University

Learning Outcomes

- Explain the MNIST image-classification task.
- Describe the role of a CNN, loss function and optimizer.
- Read the training loop in PyTorch.
- Interpret loss values and the recorded test accuracy.

Why Deep Learning Before Quantum ML?

- Quantum ML examples often borrow familiar ML components.
- Understanding tensors, training, loss and optimization makes hybrid quantum-classical models easier to follow.
- The notebook provides a classical neural-network baseline on MNIST.

MNIST and the Data Pipeline

- MNIST contains handwritten digit images.
- `ToTensor()` converts images into tensors.
- Normalization shifts/scales pixel values.
- `DataLoaders` provide batches for training and testing.

Model Definition: CNN

```
import torch
import torch.nn as nn
import torchvision.transforms as transforms
import torchvision.datasets as datasets
from torch.utils.data import DataLoader

transform = transforms.Compose([transforms.ToTensor(), transforms.Normalize((0.5,), (0.5,))])
train_data = datasets.MNIST(root='./data', train=True, download=True, transform=transform)
test_data = datasets.MNIST(root='./data', train=False, download=True, transform=transform)
train_loader = DataLoader(train_data, batch_size=64, shuffle=True)
test_loader = DataLoader(test_data, batch_size=64, shuffle=False)

class CNN(nn.Module):
    def __init__(self):
        super(CNN, self).__init__()
        self.conv1 = nn.Conv2d(1, 32, kernel_size=3, stride=1, padding=1)
        self.pool = nn.MaxPool2d(2, 2)
        self.fc1 = nn.Linear(32 * 14 * 14, 10)
    def forward(self, x):
        x = self.pool(torch.relu(self.conv1(x)))
        x = x.view(-1, 32 * 14 * 14)
```

Training Loop: What Happens Each Step

```
for images, labels in train_loader:  
    optimizer.zero_grad()  
    outputs = model(images)  
    loss = criterion(outputs, labels)  
    loss.backward()  
    optimizer.step()
```

- Reset gradients before the new batch.
- Compute predictions.
- Measure error with cross-entropy.
- Backpropagate gradients.
- Update model parameters using Adam.

Interpreting the Recorded Training Output

Epoch 1: 0.0660 Epoch 2: 0.0374 Epoch 3: 0.0177...

- Loss falls strongly in the first three recorded epochs.
- Later loss values rise in the recorded run, showing that training is not guaranteed to improve monotonically.
- The notebook reports Test Accuracy: 98.23%.

What Does 98.23% Mean?

- About 98 out of every 100 test images were classified correctly in that recorded run.
- Accuracy is computed on unseen test images.
- A strong classical baseline helps learners judge what a later quantum model is trying to achieve.

Bridge to Hybrid Quantum Models

- Classical layers can preprocess or postprocess a quantum circuit.
- A quantum layer can act as one component inside a larger trainable model.
- Lesson 12 will use exactly this hybrid idea.

Beginner Activity

- Inspect one batch of MNIST images.
- Change the batch size.
- Change the number of epochs and observe the effect on loss.
- Discuss why the test set is kept separate from training.

Lesson Summary

- A CNN maps images to class scores.
- Loss measures prediction error.
- Backpropagation supplies gradients.
- The notebook records 98.23% test accuracy.

Deep learning provides the training vocabulary used later in hybrid quantum-classical models.

Quick Check

- ① What is a batch?
- ② Why normalize images?
- ③ What does `loss.backward()` do conceptually?
- ④ Why evaluate on a separate test set?